

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA0501

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool : Scenario-Based Task (High)
Assessment Tool Weightage: 35%

Dr.A.JAYA MABEL RANI

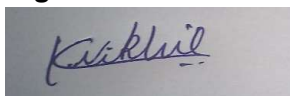
QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Given the scenario of a School Management System, identify relations and write SQL to list all students with their class teacher and grade.	BL3 – Apply	5
2	A hospital wants to track patient appointments. Design the relational schema and write SQL to find doctors with more than 10 appointments this month.	BL3 – Apply	5
3	In an e-commerce scenario, write SQL queries using sub-queries to find products that have never been ordered.	BL3 – Apply	5
4	For a university database, write relational algebra expressions to find students enrolled in both 'DBMS' and 'OS' courses.	BL3 – Apply	5
5	Given a payroll scenario, create a view showing employees whose salary is below department average and write a query to update their salary by 10%.	BL6 – Create	5
6	In a banking scenario, apply JOIN operations to retrieve customer account details, transaction history, and branch information.	BL3 – Apply	5

7	A retail store needs daily sales summary. Write SQL with GROUP BY and HAVING to report total sales per category exceeding Rs. 10,000.	BL3 – Apply	5
8	For a library system scenario, construct tuple relational calculus expressions to find members who borrowed books from all genres.	BL3 – Apply	5
9	Given an inventory management scenario, define CHECK and FOREIGN KEY constraints and demonstrate their enforcement through SQL.	BL3 – Apply	5
10	Design a relational schema for an HR system and create materialized views for monthly attendance and performance summaries.	BL6 – Create	5

Code of Conduct:

*I K.NIKHIL / 192424366 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:



RUBRICS FOR CALCULATION:

Scenario based assessment					
Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts

		address the scenario			
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

1. School Management System

Scenario Understanding:

A school stores student, class, and teacher details. We need to list each student along with their class teacher and grade.

Relations

Student(StudentID, StudentName, ClassID)

Class(ClassID, ClassName, TeacherID)

Teacher(TeacherID, TeacherName)

Grade(StudentID, Grade)

SQL Query

```
SELECT S.StudentName,  
       T.TeacherName,  
       G.Grade  
FROM Student S  
JOIN Class C  
ON S.ClassID = C.ClassID  
JOIN Teacher T  
ON C.TeacherID = T.TeacherID  
JOIN Grade G  
ON S.StudentID = G.StudentID;
```

This query joins the **Student**, **Class**, **Teacher**, and **Grade** tables to display each student's name along with their class teacher and grade. It demonstrates the use of SQL **JOIN** operations to retrieve related data.

2. Hospital Appointment System

Relational Schema

Doctor(DoctorID, DoctorName, Specialization)

Patient(PatientID, PatientName)

Appointment(AppointmentID, DoctorID, PatientID, AppointmentDate)

SQL Query

```
SELECT D.DoctorName,  
       COUNT(A.AppointmentID) AS TotalAppointments  
FROM Doctor D  
JOIN Appointment A  
ON D.DoctorID = A.DoctorID  
WHERE MONTH(A.AppointmentDate)=MONTH(CURDATE())  
AND YEAR(A.AppointmentDate)=YEAR(CURDATE())  
GROUP BY D.DoctorID, D.DoctorName  
HAVING COUNT(A.AppointmentID) > 10;
```

The relational schema organizes doctor, patient, and appointment information. The SQL query uses **GROUP BY** and **HAVING** to identify doctors who have handled more than 10 appointments in the current month.

3. E-Commerce System

Find products never ordered

```
SELECT ProductName
FROM Product
WHERE ProductID NOT IN
(
  SELECT ProductID
  FROM Orders
);
```

The SQL query uses a **subquery** to find products that have never been ordered. This helps the company identify unsold products for inventory or marketing decisions.

4. University Database

Relations

- Student(StudentID, StudentName)
- Enrollment(StudentID, CourseID)
- Course(CourseID, CourseName)

Relational Algebra

$$\text{DBMS} \leftarrow \pi_{\text{StudentID}}(\sigma_{\text{CourseName}='DBMS'}(\text{Enrollment} \bowtie \text{Course}))$$
$$\text{OS} \leftarrow \pi_{\text{StudentID}}(\sigma_{\text{CourseName}='OS'}(\text{Enrollment} \bowtie \text{Course}))$$
$$\text{Result} \leftarrow \text{DBMS} \cap \text{OS}$$

The relational algebra expression retrieves students enrolled in both **DBMS** and **OS** courses by applying **selection**, **projection**, **join**, and **intersection** operations.

5. Payroll System

Create View

```
CREATE VIEW LowSalaryEmployees AS
SELECT E.EmpID,
       E.EmpName,
       E.DepartmentID,
       E.Salary
FROM Employee E
WHERE Salary <
```

```
(  
SELECT AVG(Salary)  
FROM Employee  
WHERE DepartmentID = E.DepartmentID  
);
```

Update Salary

```
UPDATE Employee  
SET Salary = Salary * 1.10  
WHERE EmpID IN  
(  
SELECT EmpID  
FROM LowSalaryEmployees  
);
```

A **view** is created to identify employees earning below their department's average salary. The update query then increases their salary by **10%**, showing how views simplify data management.

6. Banking System

Relations

- Customer(CustomerID, CustomerName)
- Account(AccountNo, CustomerID, BranchID)
- Transaction(TransactionID, AccountNo, Amount, TransactionDate)
- Branch(BranchID, BranchName)

SQL Query

```
SELECT C.CustomerName,  
       A.AccountNo,  
       T.TransactionID,  
       T.Amount,  
       T.TransactionDate,  
       B.BranchName  
FROM Customer C  
JOIN Account A  
ON C.CustomerID=A.CustomerID  
JOIN Transaction T  
ON A.AccountNo=T.AccountNo  
JOIN Branch B  
ON A.BranchID=B.BranchID;
```

The query combines **Customer**, **Account**, **Transaction**, and **Branch** tables using **JOINS** to provide complete banking information, including account details, transaction history, and branch information.

7. Retail Store Sales Summary

```
SELECT Category,  
       SUM(SalesAmount) AS TotalSales  
FROM Sales  
GROUP BY Category  
HAVING SUM(SalesAmount) > 10000;
```

The SQL query groups sales by category using **GROUP BY** and filters categories with total sales exceeding **₹10,000** using the **HAVING** clause. This helps analyze high-performing product categories.

8. Library System

Tuple Relational Calculus

```
{ M.MemberName |  
  Member(M) AND  
  ∀G (Genre(G) →  
    ∃B ∃L  
    (Book(B) AND  
     Borrow(L) AND  
     L.MemberID=M.MemberID AND  
     L.BookID=B.BookID AND  
     B.Genre=G.GenreName))  
}
```

This returns members who have borrowed books from **every genre**.

The tuple relational calculus expression identifies members who have borrowed books from **every available genre**. It demonstrates declarative database querying using relational calculus.

9. Inventory Management

Table Creation

```
CREATE TABLE Supplier(  
  SupplierID INT PRIMARY KEY,  
  SupplierName VARCHAR(50)  
);
```

```
CREATE TABLE Product(  
  ProductID INT PRIMARY KEY,  
  ProductName VARCHAR(50),  
  Price DECIMAL(10,2)  
  CHECK (Price > 0),  
  SupplierID INT,  
  FOREIGN KEY (SupplierID)  
  REFERENCES Supplier(SupplierID)
```

);

Constraint Enforcement

Valid Insert

```
INSERT INTO Supplier VALUES(1,'ABC Traders');
```

```
INSERT INTO Product
```

```
VALUES(101,'Keyboard',800,1);
```

Invalid Insert (CHECK fails)

```
INSERT INTO Product
```

```
VALUES(102,'Mouse',-300,1);
```

Invalid Insert (FOREIGN KEY fails)

```
INSERT INTO Product
```

```
VALUES(103,'Monitor',7000,5);
```

The **CHECK** constraint ensures valid product prices, while the **FOREIGN KEY** constraint maintains valid supplier references. The SQL examples demonstrate how these constraints enforce data integrity.

10. HR System

Relational Schema

Employee(EmplID, EmpName, DeptID)

Attendance(EmplID, AttendanceDate, Status)

Performance(EmplID, Month, Rating)

Materialized View – Monthly Attendance

```
CREATE MATERIALIZED VIEW MonthlyAttendance AS
```

```
SELECT EmplID,
```

```
    MONTH(AttendanceDate) AS Month,
```

```
    COUNT(*) AS TotalDaysPresent
```

```
FROM Attendance
```

```
WHERE Status='Present'
```

```
GROUP BY EmplID,
```

```
    MONTH(AttendanceDate);
```

Materialized View – Performance Summary

```
CREATE MATERIALIZED VIEW PerformanceSummary AS
```

```
SELECT EmplID,
```

```
    AVG(Rating) AS AverageRating
```

```
FROM Performance
```

```
GROUP BY EmplID;
```

The relational schema stores employee, attendance, and performance data.

Materialized views are created to maintain monthly attendance and performance summaries, enabling faster report generation.

